Atelier 4 : Git et GitHub, travail collaboratif (réponses)

Stagiaire : Ali EL Habib · OFPPT Tanger · Développement Digital
Dépôt : `atelier-collaboratif` · branche principale : `main`

> **Remarque sur la réalisation.** L'atelier est prévu en binôme. Je l'ai réalisé seul en jouant les deux rôles : un dossier local par développeur (`dev1-atelier-collaboratif` et `dev2-atelier-collaboratif`), avec le même compte GitHub. L'invitation d'un collaborateur (Partie 3) demande un deuxième compte, donc cette étape est décrite ci-dessous mais n'a pas pu être exécutée. Le dépôt a été cloné en **HTTPS** au lieu de SSH (même résultat, seule l'URL change).

Partie 1 : dépôt distant et première version (Développeur 1)

```
# dépôt "atelier-collaboratif" créé vide sur GitHub (sans README, .gitignore ni licence)
git clone https://github.com/alilhbib20007-sys/atelier-collaboratif.git
cd atelier-collaboratif
git status
git branch
git remote -v

# création de index.html
git status
git add index.html
git status
git commit -m "Ajouter la page d'accueil"
git branch -M main
git push origin main
```

Quel est le rôle de `git clone` ? Copier un dépôt distant sur la machine locale : les fichiers, tout l'historique et les références des branches distantes. Il configure aussi automatiquement le dépôt distant sous le nom `origin`.

Quel dossier a été créé sur votre machine ? Un dossier `atelier-collaboratif` (du nom du dépôt), qui contient le dossier caché `.git`.

Que représente `origin` dans le résultat de `git remote -v` ? C'est le nom par défaut donné au dépôt distant cloné (ici le dépôt GitHub). `git remote -v` affiche son URL pour `fetch` (récupérer) et `push` (envoyer).

Quelle est la différence entre le dépôt local et le dépôt distant ? Le dépôt local est sur mon ordinateur : je peux y travailler et faire des commits sans connexion. Le dépôt distant est hébergé sur GitHub : il sert de référence commune à l'équipe. Les deux ne se synchronisent que quand on le demande (`git push` pour envoyer, `git pull` / `git fetch` pour récupérer).

Pourquoi exécuter `git add` avant `git commit` ? Parce que `git commit` n'enregistre que ce qui se trouve dans la zone de staging. `git add` sert à choisir les modifications à inclure.

Que représente le commit créé ? Une version enregistrée du projet, ici la première avec la page d'accueil. Il a un identifiant unique (hash), un auteur, une date et un message.

Quel est le rôle de `git push` ? Envoyer les commits locaux de la branche vers le dépôt distant (`origin`).

Que constatez-vous sur GitHub après le push ? Le dépôt n'est plus vide : on voit le fichier `index.html` , la branche `main` et le commit « Ajouter la page d'accueil ».

Partie 2 : branche `contact` (Développeur 1)

```
git checkout -b contact
git branch                          # * contact
# création de contact.html
git add contact.html
git commit -m "Ajouter la page contact"
git push --set-upstream origin contact
```

La branche `contact` apparaît ensuite sur GitHub.

Partie 3 : inviter le deuxième développeur (Développeur 1)

Sur GitHub : **Settings → Collaborators → Add people**, saisir le compte du Développeur 2, qui accepte l'invitation reçue par email. Il obtient alors le droit de pousser sur le dépôt.

(Non exécuté ici : un seul compte GitHub disponible.)

Partie 4 : branche `promotion` (Développeur 2)

```
git clone https://github.com/alilhbib20007-sys/atelier-collaboratif.git dev2-atelier-
collaboratif
cd dev2-atelier-collaboratif
git branch          # * main
git branch -r       # origin/HEAD -> origin/main, origin/contact, origin/main
git remote -v

git checkout main
git checkout -b promotion
# création de promotion.html
git add promotion.html
git commit -m "Ajouter la page promotion"
git push --set-upstream origin promotion
```

Constats : la branche locale est `main` ; `origin/main` et `origin/contact` sont présentes en distant ; il n'y a **pas** de branche locale `contact` (le clone ne crée qu'une branche locale, la branche par défaut) ; `index.html` est bien présent.

Partie 5 : Pull Requests et intégration

- **PR « Ajouter la page Contact »** : `contact` → `main`, puis fusionnée (*Merge pull request*).
- **PR « Ajouter la page Promotion »** : `promotion` → `main`, puis fusionnée.

Synchronisation des deux dépôts locaux :

```
git checkout main
git pull origin main
```

La branche `main` contient maintenant `index.html`, `contact.html` et `promotion.html`.

---

7. Questions de synthèse

1. Différence entre dépôt local et dépôt distant ? Le local est la copie de travail sur la machine du développeur (avec son propre historique) ; le distant est la copie partagée hébergée sur GitHub, qui sert de point de rencontre de l'équipe.

2. Pourquoi chaque développeur travaille-t-il sur une branche distincte ? Pour développer sa fonctionnalité de façon isolée, sans casser la branche principale ni gêner le travail de l'autre. Les changements ne rejoignent `main` qu'une fois terminés et relus.

3. Rôle de `origin` dans `git push origin contact` ? Il désigne le dépôt distant de destination (le dépôt GitHub configuré lors du clone). `contact` est la branche à envoyer.

4. Pourquoi `--set-upstream` lors du premier push d'une branche ? Pour lier la branche locale à la branche distante du même nom. Ensuite, un simple `git push` ou `git pull` suffit, sans répéter `origin contact`, et `git status` indique l'avance ou le retard par rapport au distant.

5. Différence entre `git push` et une Pull Request ? `git push` est une commande Git qui envoie des commits vers le dépôt distant. Une Pull Request est une fonctionnalité de GitHub : une demande pour fusionner une branche dans une autre, avec relecture, commentaires et validation avant la fusion.

6. Une Pull Request fusionne-t-elle automatiquement la branche ? Non. Créer une PR ouvre seulement une demande. La fusion n'a lieu que lorsque quelqu'un clique sur **Merge pull request** (après relecture). La PR peut aussi être refusée ou fermée sans fusion.

7. Pourquoi récupérer la branche principale après une fusion ? La fusion a été faite sur GitHub : le `main` local de chaque développeur est donc en retard. `git pull origin main` met le dépôt local à jour pour repartir de la dernière version et éviter des conflits plus tard.

8. Combien de branches ont été créées et quelles sont leurs fonctions ? Trois branches : - `main` (ou `master`) : branche principale, version stable du site ; - `contact` : développement de la page Contact (Développeur 1) ; - `promotion` : développement de la page Promotion (Développeur 2).